

```

// [support.contract.control.semset.access], accessors
constexpr bool contains(evaluation_semantic) const;
constexpr size_t size() const;
constexpr bool empty() const;

// [support.contract.control.semset.mod], modifiers
constexpr void clear();
constexpr void insert(evaluation_semantic);
constexpr void insert(initializer_list<evaluation_semantic>);
constexpr void remove(evaluation_semantic);

// [support.contract.control.semset.setops], set operations
constexpr evaluation_semantic_set& operator&=(const evaluation_semantic_set&);
constexpr evaluation_semantic_set& operator|=(const evaluation_semantic_set&);
constexpr evaluation_semantic_set operator~() const;

friend constexpr evaluation_semantic_set operator&(const evaluation_semantic_set&,
                                                    const evaluation_semantic_set&);
friend constexpr evaluation_semantic_set operator|(const evaluation_semantic_set&,
                                                    const evaluation_semantic_set&);

// [support.contract.control.semset.cmp], comparisons
friend constexpr bool operator==(const evaluation_semantic_set&,
                                const evaluation_semantic_set&);

// [support.contract.control.semset.factory], factory functions
static constexpr evaluation_semantic_set all();
static constexpr evaluation_semantic_set none();
};
}

```

- 1 An object of type `evaluation_semantic_set` represents a set of `evaluation_semantic` values.

17.10.(4 + a).11.2 Constructors [support.contract.control.semset.cons]

17.10.(4+a).11.2 Constructors [support.contract.control.semset.cons]

```
constexpr evaluation_semantic_set();
```

- 1 *Postconditions:* `contains(s)` is false for every enumerator `s` of `evaluation_semantic`.

```
constexpr evaluation_semantic_set(initializer_list<evaluation_semantic> ilist);
```

- 2 *Postconditions:* For each enumerator `s` of `evaluation_semantic`, `contains(s)` is true if and only if `s` is an element of `ilist`.

17.10.(4 + a).11.3 Accessors [support.contract.control.semset.access]

17.10.(4+a).11.3 Accessors [support.contract.control.semset.access]

```
constexpr bool contains(evaluation_semantic s) const;
```

- 1 *Returns:* true if `s` is an element of the set; otherwise, false.

```
constexpr size_t size() const;
```

- 2 *Returns:* The number of elements in the set.

3

3

[support.contract.control.semset.mod]

[support.contract.control.semset.mod]

1

1

2

2

3

3

4

4

[support.contract.control.semset.setops]

[support.contract.control.semset.setops]

1

1

2

2

3

3

4

4

5

5

6

6

7

7

17.10.(4 + a).11.6 Comparisons [support.contract.control.semset.cmp]

17.10.(4+a).11.6 Comparisons [support.contract.control.semset.cmp]

```
friend constexpr bool operator==(const evaluation_semantic_set& lhs,
                                const evaluation_semantic_set& rhs);
```

1 *Returns:* true if, for every enumerator *s* of *evaluation_semantic*,
 lhs.contains(s) == rhs.contains(s); otherwise, false.

17.10.(4 + a).11.7 Factory functions [support.contract.control.semset.factory]

17.10.(4+a).11.7 Factory functions [support.contract.control.semset.factory]

```
static constexpr evaluation_semantic_set all();
```

1 *Returns:* An *evaluation_semantic_set* *r* such that *r.contains(s)* is true for
 every enumerator *s* of *evaluation_semantic*.

```
static constexpr evaluation_semantic_set none();
```

2 *Returns:* *evaluation_semantic_set()*.

17.10.(4 + a).12 Evaluation semantic classification [support.contract.control.semfun]

17.10.(4+a).12 Evaluation semantic classification [support.contract.control.semfun]

```
constexpr bool is_checked(evaluation_semantic s);
```

1 *Returns:* true if *s* is *evaluation_semantic::observe*, *evaluation_-*
 semantic::enforce, or *evaluation_semantic::quick_enforce*; otherwise,
 false.

```
constexpr bool is_unchecked(evaluation_semantic s);
```

2 *Returns:* *!is_checked(s)*.

```
constexpr bool is_terminating(evaluation_semantic s);
```

3 *Returns:* true if *s* is *evaluation_semantic::enforce* or *evaluation_-*
 semantic::quick_enforce; otherwise, false.

```
constexpr bool is_continuing(evaluation_semantic s);
```

4 *Returns:* *!is_terminating(s)*.

17.10.(4 + a).13 Class template *dimension_list* [support.contract.control.dimlist]

17.10.(4+a).13 Class template *dimension_list* [support.contract.control.dimlist]

```
namespace std::contracts::labels {
    template<class... Dims>
        struct dimension_list {};
}
```